HDOC Day-50

****

Write a C program that reads the size and elements of a variable-length integer array, nums[]. Pass the array to a function named majority_element() using its address. Use pointers to pass the array between the calling function and majority_element(). The function majority_element() must:

- Count the occurrences of each distinct element in nums[].
- Find the element that appears strictly > [size / 2] times.
- Display the majority element if it exists.
- Display -1 if no majority element exists.

Note: The majority element is not necessarily the element with the highest frequency. It must occur strictly > [size / 2] times.

1. Problem Understanding

The program reads the size and elements of a variable-length integer array nums[]. It passes the array to majority_element() using its address. The function counts the occurrences of each element and returns an element only when its count is strictly greater than size / 2. If no such element exists, it returns -1.

2. Standard Test Cases

Test Case	Size (n)	Array Input	Expected Output	Explanation
1	5	3 3 2 3 3	Majority element: 3	3 occurs 4 times; 4 > 5/2 = 2
2	7	2 2 1 1 1 2 2	Majority element: 2	2 occurs 4 times; 4 > 7/2 = 3
3	6	1 2 3 2 3 3	-1	3 occurs 3 times; 3 is not > 6/2 = 3
4	1	9	Majority element: 9	9 occurs once; 1 > 1/2 = 0
5	4	5 5 2 2	-1	Highest count is 2; 2 is not > 4/2 = 2

3. Algorithm

Main Algorithm

1. Start.
2. Read the size of the array.
3. Create a variable-length integer array nums[size].
4. Read all array elements.
5. Pass nums and its size to majority_element().
6. Display the majority element if it exists; otherwise display -1.

Function: majority_element()

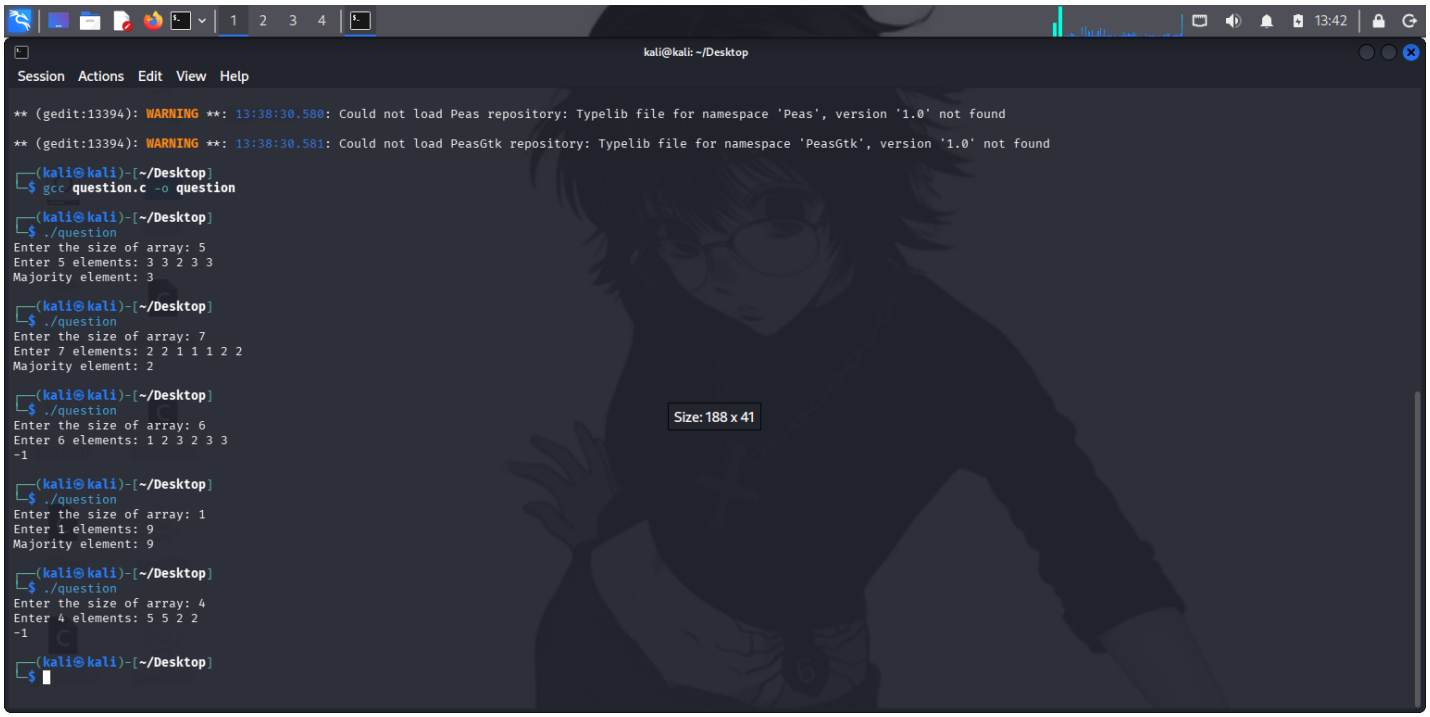
1. Receive the array using an integer pointer and receive its size.
2. For each element, initialize count to 0.
3. Compare it with every element in the array and count its occurrences.
4. If count is strictly greater than size / 2, return that element.
5. If no element satisfies the condition, return -1.

4. C Program

```
#include <stdio.h>
int majority_element(int *nums, int size)
{
   int i, j, count;
   for(i = 0; i < size; i++)
   {
      count = 0;
      for(j = 0; j < size; j++)
      {
         if(*(nums + i) == *(nums + j))
         {
            count++;
         }
      }
      if(count > size / 2)
      {
         return *(nums + i);
      }
   }
   return -1;
}
int main()
{
   int size, i, result;
   printf("Enter the size of array: ");
   scanf("%d", &size);
   int nums[size];
   printf("Enter %d elements: ", size);
   for(i = 0; i < size; i++)
   {
      scanf("%d", &nums[i]);
   }
```

```
    result = majority_element(nums, size);
    if(result == -1)
        printf("-1\n");
    else
        printf("Majority element: %d\n", result);
    return 0;
}
```

5. Compilation and Execution Evidence



6. Verification of All Test Cases

Test Case	Size (n)	Array Input	Actual Result	Expected Result	Status
1	5	3 3 2 3 3	Majority element: 3	Majority element: 3	Pass
2	7	2 2 1 1 1 2 2	Majority element: 2	Majority element: 2	Pass
3	6	1 2 3 2 3 3	-1	-1	Pass
4	1	9	Majority element: 9	Majority element: 9	Pass
5	4	5 5 2 2	-1	-1	Pass